U.S. Airline Flight Delay Analysis Pipeline

Final Project Report

Student: Fabricio Pons Samano
Course: Data-Intensive Systems — Final Project
Date: May 10, 2026
Repository: https://github.com/FabricioPons/flight-delay-pipeline
Live Dashboard: https://datastudio.google.com/reporting/53ef7ced-41f6-4e30-893b-c9a9654fce25

1. Project Overview and Goals

This project set out to build a complete, cloud-native data pipeline on Google Cloud Platform that could tell a meaningful story about U.S. airline flight delays from 2019 through 2025. To make the analysis richer, I brought in daily weather observations from the NOAA Global Surface Summary of the Day dataset as a second source. The end result is two concrete deliverables that anyone could use.

1. An interactive Looker Studio dashboard that lets you explore how airline, route, time of day, and weather conditions each play into on-time performance.
2. Two BigQuery ML classifiers, a logistic regression baseline and a boosted-tree ensemble built on XGBoost, that predict whether a given flight will arrive more than 15 minutes late.

One thing I was deliberate about from the start was reproducibility. The entire pipeline runs from a single environment variable (`PROJECT_ID`) and four shell scripts, so anyone could spin it up fresh. On the cost side, every phase together came in under **$11** of GCP free credit, which required some discipline but was very doable.

2. Dataset Description

2.1 BTS On-Time Performance (Primary)

The backbone of the project is the Bureau of Transportation Statistics TranStats dataset. I pulled 84 monthly zip files spanning January 2019 through December 2025, which came out to roughly 2.4 GB compressed and about 14 GB once unzipped. After cleaning, the dataset held 45,763,492 individual flight records. I kept 31 columns, including the core identifiers like airline and route, delay and cancellation fields, the five BTS delay cause categories (carrier, weather, NAS, security, and late aircraft), and a handful of derived fields like departure hour and a binary delayed flag.

2.2 NOAA GSOD (Secondary)

For weather context, I used the NOAA Global Surface Summary of the Day dataset, which is already available as a BigQuery public table so there was no ingestion step needed. It provides daily aggregates per weather station including mean wind speed, precipitation, visibility, and temperature. After filtering down to active U.S. stations, I ended up with 18,176 unique airport-to-station mappings.

2.3 FAA Airports (Tertiary)

The FAA airports table from BigQuery's public data served two purposes: attaching latitude and longitude to origin airport codes for the geographic dashboard panel, and anchoring the haversine nearest-neighbor lookup that connects each airport to its closest NOAA weather station.

3. Pipeline Workflow and Services Used

The pipeline moves data through six stages. Raw BTS zip files land in Cloud Storage first, then PySpark on Dataproc handles all the heavy cleaning and joining. The cleaned, enriched dataset goes into BigQuery, where SQL views power the dashboard and BigQuery ML handles the modeling. The visualization layer sits entirely in Looker Studio on top of those views.

Stage	Service	Purpose
Ingestion	Cloud Storage	Store raw BTS zip files partitioned by year/month
Transformation	Dataproc + PySpark 3.5 (image 2.2)	Parse zips, coerce types, derive fields, join flights x weather
Warehouse	BigQuery	Partitioned + clustered analytical table; 5 reporting views; 3 ML output tables
Analytics	BigQuery SQL	Five dashboard-backing views over flights_weather_enriched
ML	BigQuery ML	Logistic regression and boosted-tree classifiers, evaluated on a 2025 holdout
Visualization	Looker Studio	Two-page interactive dashboard (5 analytics panels + 3 ML panels)

For the cluster, I settled on a 1 x e2-standard-2 master with 2 x e2-standard-4 workers for a total of 10 vCPUs. I also set a 1-hour idle auto-delete so the cluster would not sit running between job submissions and rack up unnecessary charges.

4. Data Processing Steps

4.1 BTS Cleaning (clean_flights.py)

This job reads every monthly zip from Cloud Storage using `sc.binaryFiles`, unpacks each archive with Python's zipfile module inside a distributed flatMap, and then trims the schema down to the 31 columns I actually needed. After coercing data types and writing the output as Parquet partitioned by year and month, the job produced 84 output partitions covering roughly 46 million rows.

4.2 Airport to Weather-Station Mapping (airport_station_map.py)

Connecting each airport to its closest NOAA station required a haversine nearest-neighbor join. I filtered the NOAA stations table down to U.S. stations with a valid WBAN code that were still active as of January 2019, then collapsed duplicates by WBAN to clean up code revisions over the years. From there I cross-joined airports and stations within a +/-0.75-degree bounding box, computed the actual haversine distance, kept only matches within 50 km, and selected the single closest station per airport. The result was 18,176 airport-to-station pairs.

4.3 Flight x Weather Join (join_flights_weather.py)

This job brings flights and weather together by joining on station ID and date for both the origin and destination airports. One wrinkle: Spark's column resolver could not bind GSOD's native `date` column, so I had to reconstruct the flight date from the separate year, month, and day fields using `to_date(concat_ws("-", ...))`. The enriched output was written to BigQuery partitioned by flight date and clustered by airline, origin, and destination.

- Final row count: 45,763,492.
- Origin weather coverage came out to 94.4%. Major hubs like ATL, DFW, DEN, ORD, and LAX all landed between 93.6% and 95.5%.

4.4 BigQuery View Materialization (create_tables.sql)

With the enriched table in place, I materialized five SQL views to back the dashboard panels:

View	Granularity	Powers
v_monthly_ontime_trend	One row per month	Time-series panel
v_delay_cause_breakdown	One row per airline x month	100% stacked bar
v_airport_delay_heatmap	One row per origin airport	Geo bubble map
v_weather_correlation	One row per wind/precip/visibility bucket	Pivot heatmap
v_airline_kpi	One row per airline	Conditional-format table

Three additional tables back the ML page: m_eval_metrics, m_confusion_matrix, and m_feature_importance, each materialized from the corresponding BigQuery ML evaluation functions. A fourth table, m_model_comparison, holds the side-by-side metrics so both models can be compared directly in the dashboard.

5. Results

5.1 Headline Numbers

At the highest level, the pipeline processed 45,763,492 flights covering the full 2019 through 2025 period. Weather data from NOAA attached successfully to 94.4% of those flights on the origin side. Across the entire seven-year window the overall on-time rate came out to approximately 78.7%.

5.2 Airline Leaderboard (On-Time %)

Airline	On-Time %
Delta (DL)	83.3%
YX	81.2%
SkyWest (OO)	81.1%
Alaska (AS)	78.9%
United (UA)	78.9%
Southwest (WN)	78.2%
OH	78.0%
American (AA)	76.1%
JetBlue (B6)	71.4%

5.3 COVID-19 Impact (Cancellation Rate)

The pandemic's impact on the network is hard to miss in the data. The 2019 baseline cancellation rate was 1.82%. That number jumped to 5.99% in 2020, peaking sharply between March and June in a way that is clearly visible in the monthly time-series panel as a vertical drop in on-time rate followed by a gradual multi-month recovery. By 2021 the cancellation rate had already returned to 1.72%, essentially back to pre-pandemic levels.

5.4 Weather-Correlated Delay Rate

The weather pivot heatmap revealed something I did not expect going in. As origin conditions get worse, delay rates climb steadily up to around 30 knots of wind. But above that threshold, the delay rate actually falls sharply. The reason is that at that point airlines stop delaying flights and start cancelling them outright, so the delayed-flight pool shrinks while the cancelled-flight pool grows. The table below shows this pattern for poor visibility conditions:

Wind Speed	Delay Rate (Poor Visibility < 3 mi)
0-5 kt	24.5%
5-10 kt	30.2%
10-20 kt	39.6% (peak)
20-30 kt	31.3%
30+ kt	5.3% (cancellation pool)

This dual-mode response, where airlines delay until a breaking point and then switch to cancellation, was one of the more interesting findings to come out of the analysis.

5.5 Logistic Regression Baseline (delay_clf)

The logistic regression was trained on flights from January 2019 through December 2024, roughly 38 million rows, with the full year of 2025 held out for evaluation. The model used auto class weights to handle the class imbalance between on-time and delayed flights. Features included the airline, origin, destination, departure hour, day of week, month, distance, and the four origin weather fields (wind, precipitation, visibility, and temperature).

Metric	Value
Accuracy	0.5611
Precision	0.2958
Recall	0.6347
F1	0.4035
Log-loss	0.6892
ROC-AUC	0.6152

Top features by ML.GLOBAL_EXPLAIN:

Feature	Attribution
dep_hour	0.042
reporting_airline	0.023
origin	0.023
dest	0.019
month	0.009

5.6 Boosted-Tree Comparison (delay_clf_boosted)

The boosted tree used the same feature set and the same 2019-2024 training window, so the comparison against the logistic baseline is apples to apples. I set max_iterations to 50, max_tree_depth to 8, learn_rate to 0.1, and subsample to 0.8.

Metric	Logistic	Boosted	Delta
Accuracy	0.5611	0.5874	+0.0263
Precision	0.2958	0.3164	+0.0206
Recall	0.6347	0.6582	+0.0235
F1	0.4035	0.4273	+0.0238
Log-loss	0.6888	0.6827	-0.0061 (better)
ROC-AUC	0.6152	0.6465	+0.0313

The boosted tree improved every metric and lifted ROC-AUC by 0.031 absolute. That gain is real, but it is also modest, which confirms that the feature set itself is the binding constraint rather than the choice of algorithm.

The feature ranking from ML.GLOBAL_EXPLAIN shifted noticeably between the two models:

Feature	Attribution	Rank vs. Logistic
dep_hour	0.030	#1 in both models
visibility	0.015	jumped from low to #2
month	0.010	up
temperature	0.010	up
distance	0.005	similar
reporting_airline	0.004	dropped from #2 to #7
origin	0.0004	dropped from #3 to #9

5.7 Interpretation of Model Results

A ROC-AUC of 0.62 on the logistic baseline means the model has weak but real predictive signal. It is doing something, just not a lot. Two structural limitations explain why the ceiling is so low.

3. Daily weather resolution is simply too coarse. The events that actually delay individual flights, things like a line storm passing through or a sudden wind gust, happen on a timescale of minutes or hours. Daily averages from GSOD smooth all of that out. Hourly METAR observations would almost certainly help here.
4. The biggest single driver of delays in the BTS data is late aircraft propagation, which accounts for roughly 30% of all delay minutes. The problem is that capturing this requires tracking an aircraft's tail number across consecutive legs, and that information was not included in the feature set. Adding rotation features would likely be the highest-leverage improvement available.

The boosted tree's modest improvement over the logistic baseline confirms that some nonlinear interactions do exist in the data, and the jump in visibility from a low-ranked feature to the number two slot is the clearest example of that. But neither model can escape the ceiling imposed by what the features actually measure. Changing the algorithm without changing the features will only get you so far.

6. Visualizations and Dashboard

Live dashboard: https://datastudio.google.com/reporting/53ef7ced-41f6-4e30-893b-c9a9654fce25

The Looker Studio dashboard is split into two pages with shared filters at the top of page 1: a date range picker, an airline selector, and an airport code filter. Page 1 covers the analytics story and page 2 covers the machine learning results.

Page 1 — Analytics (5 Panels)

5. Monthly on-time rate (2019-2025): a line series where the COVID-19 dip between March and June 2020 is immediately visible to anyone looking at it.
6. Delay cause breakdown by airline: a 100% stacked bar chart showing the share of carrier, weather, NAS, security, and late aircraft delay minutes broken out by reporting airline.
7. Airport delay heatmap: a Google Maps bubble map where bubble size reflects flight volume and color reflects mean arrival delay, built on a calculated airport_geo field that combines FAA latitude and longitude into a format Looker can geocode reliably.
8. Weather vs. delay rate: a pivot heatmap crossing wind buckets against visibility buckets, with each cell colored by the delay rate inside that combination.
9. Airline KPI table: a sortable, conditional-formatted table showing total flights, mean arrival delay, on-time percentage, and cancellation percentage per airline.

Page 2 — ML (3 Panels + Comparison Table)

10. Model comparison table: a side-by-side view of all evaluation metrics for both models, pulled from the m_model_comparison BigQuery table.
11. Confusion matrix: a pivot heatmap built on the 2025 holdout set of roughly 7 million flights, filterable by model.
12. Feature importance: a horizontal bar chart from ML.GLOBAL_EXPLAIN showing which features each model weighted most heavily.

A static PDF export of the complete dashboard is also included in the repository at dashboard/Final_482.pdf.

7. Challenges Encountered and Resolutions

13. **Challenge 1:** Maven egress blocked from Dataproc workers.

    The default spark.jars.packages resolution could not reach repo1.maven.org from inside the cluster network. I fixed this by switching every BigQuery-touching job to the Spark BigQuery connector that ships pre-installed with Dataproc 2.2 under /usr/lib/spark/external/, which meant dropping the --jars flag entirely.

14. **Challenge 2:** CPU quota and zone capacity.

    New GCP projects default to 12 vCPUs per region, and us-central1-a had no available n2 capacity when I tried to provision. The fix was downsizing to 10 vCPUs using e2-class machines and switching to Dataproc auto-zone so the cluster could land wherever capacity was available.

15. **Challenge 3:** FAA table column names did not match expectations.

    My initial code assumed columns named iata_code, latitude_deg, and longitude_deg. The actual schema uses faa_identifier, latitude, and longitude. For U.S. commercial airports, faa_identifier and IATA code are the same value, so the join still worked cleanly after renaming.

16. **Challenge 4:** Spark could not resolve GSOD's native date column.

    I worked around this by assembling the flight date from the separate year, month, and day columns using to_date(concat_ws("-", ...)) rather than relying on the date field that Spark refused to bind.

17. **Challenge 5:** Stale stations dominated the nearest-neighbor join.

    Initial weather coverage was only 17%, with major hubs like ORD and DEN showing 0%. The culprit was that the NOAA stations table retains decommissioned entries going back decades, and some of those old stations happened to be geographically close to major airports. The fix was collapsing the stations table by WBAN and filtering to stations active as of January 2019. Coverage immediately jumped from 17% to 94.4%.

18. **Challenge 6:** Looker Studio summed pre-aggregated rates incorrectly.

    The on-time rate panel was initially summing 84 monthly rate values and displaying numbers over 60. I corrected this by switching every rate metric from SUM to AVG and setting the field types to Percent.

19. **Challenge 7:** IATA codes did not geocode reliably in the bubble map.

    Looker silently dropped airports whose codes its geocoder could not resolve. The fix was adding a calculated field, airport_geo, that concatenates the FAA latitude and longitude into a comma-separated string and sets the type to Geo with Latitude and Longitude, bypassing the geocoder entirely.

8. Lessons Learned

- Cloud cost discipline is worth taking seriously.

  Auto-deleting the Dataproc cluster after one hour of idle time and choosing e2-class machines kept the whole project under $11. It is tempting to leave a cluster running between job submissions, but resisting that habit stretched the budget roughly five times further.

- Read the image release notes before fighting egress.

  The hours I spent trying to fix spark.jars.packages could have been avoided entirely. The Dataproc 2.2 release notes explicitly mention the bundled Spark-BigQuery connector. Checking documentation first would have saved a lot of frustration.

- Row counts alone do not tell you if a join is working.

  The nearest-neighbor join returned 18,176 rows and looked fine on the surface. Weather coverage was actually only 17%. The real validation has to be downstream, and the fix

ended up having nothing to do with SQL and everything to do with understanding how NOAA stations are maintained over time.

- Start with logistic regression before anything fancier.

  The logistic baseline immediately showed which features carry real signal, departure hour, airline, and route, and which ones do not, which is daily weather averages in this case. That clarity shaped how I interpreted the boosted tree results and saved time I might have spent tuning a complex model before understanding the data.
- Every rate metric in Looker connected to a BigQuery view needs to be checked.

  The default aggregation Looker applies is SUM, and pre-aggregated rate fields should almost always use AVG. It is a quick setting to change once you know to look for it, but it is easy to miss.
- Dashboard iteration goes faster than you expect.

  Once the BigQuery views were in place, building the full two-page dashboard took about two evenings. Most of that time was fixing Looker's default behavior on aggregations and geocoding, not actually designing the layout.

9. Potential Next Steps

- Switch from daily GSOD weather to hourly METAR observations from the NOAA Integrated Surface Database. Daily averages smooth out the events that actually delay flights, and hourly data would give the model something much closer to the real signal.
- Add tail-number rotation features. Knowing the delay status of the prior leg flown by the same aircraft before each flight is probably the single most valuable piece of information missing from the current feature set.
- Build a streaming ingestion pipeline using Cloud Functions so new monthly BTS files get picked up and processed automatically as they are published, instead of running the batch script manually.
- Schedule the full pipeline with Cloud Composer or Airflow so it can be triggered on demand without anyone needing to spin up a cluster by hand.
- Add row-level security in Looker Studio so airline-specific stakeholders only see data filtered to their own carrier.
- Run AUTOML_CLASSIFIER in BigQuery ML to get an automated hyperparameter search benchmark and see how much headroom remains above the boosted tree.

Appendix A — Reproducing the Pipeline

The entire pipeline can be reproduced from scratch with four shell scripts. Setting the PROJECT_ID environment variable is the only manual step required before running them in order:

```
export PROJECT_ID=flight-delay-pipeline-494116

# 1. Bootstrap (APIs, buckets, BQ dataset)
```

```
bash scripts/01_gcp_bootstrap.sh

# 2. Ingest BTS zips into GCS (~30 min)
bash scripts/02_download_bts.sh

# 3. Spin up Dataproc cluster
bash scripts/03_create_dataproc.sh

# 4. Submit clean / map / join PySpark jobs (~50 min)
bash scripts/04_submit_jobs.sh

# 5. Materialize BigQuery views and ML models
bq query --use_legacy_sql=false < sql/create_tables.sql
bq query --use_legacy_sql=false < sql/bqml_delay_model.sql
bq query --use_legacy_sql=false < sql/bqml_eval_tables.sql
bq query --use_legacy_sql=false < sql/bqml_boosted_model.sql
bq query --use_legacy_sql=false < sql/bqml_compare_models.sql
```

Appendix B — Cost Summary

Service	Approx. Cost
Cloud Storage (raw + processed, 1 month)	$0.30
Dataproc (cluster active ~6 hr cumulative)	$6.00
BigQuery (queries, view materialization)	$1.00
BigQuery ML (logistic + boosted training)	$3.00
Total	< $11

Every dollar of this was covered by the GCP $300 free credit.